E-COMMERCE SALES & CUSTOMER ANALYTICS

DATA ENGINEERING ASSESSMENT — QUESTION PAPER

Scenario: You are a Data Engineer at RetailBridge, an e-commerce platform processing high-volume orders across products, customers, payments and delivery timelines. Build an end-to-end data pipeline using HDFS, Hive, Alteryx, Spark SQL, Kafka and Spark Structured Streaming. Execute every task hands-on, document commands and outputs, and submit a Word document as evidence.

Dataset Schema

Orders.csv: order_id, customer_id, order_status, order_purchase_timestamp, order_approved_at, order_delivered_timestamp, order_estimated_delivery_date

Customers.csv: customer_id, customer_zip_code_prefix, customer_city, customer_state

OrderItems.csv: order_id, product_id, seller_id, price, shipping_charges

Payments.csv: order_id, payment_sequential, payment_type, payment_installments, payment_value

SECTION 1 — HDFS [6 Marks]

a. Create HDFS directories /user/clouduser/ecommerce/raw/ and /user/clouduser/ecommerce/processed/.

b. Upload orders.csv to raw/orders/ using block size 256 MB and replication factor 1.

c. Upload order_items.csv and Payments.csv to their respective raw/ directories using block size 128 MB.

d. Upload customers.csv to raw/customers/ using the default block size.

e. Check disk usage of the entire /user/clouduser/ecommerce directory.

f. Copy (do NOT move) orders.csv from raw/orders/ to processed/.

SECTION 2 — HIVE [8 Marks]

a. Create Hive database ecomDB_.

b. Create an EXTERNAL table orders_raw pointing to /user/clouduser/ecommerce/raw/orders/ with fields matching orders.csv.

c. Create a MANAGED partitioned table orders_part with all orders columns, partitioned by order_year INT and order_month INT.

d. Enable dynamic partitioning and load data from orders_raw into orders_part, deriving order_year and order_month from order_purchase_timestamp.

e. Display the created PARTITIONS on orders_part.

f. Compute the total number of orders in each order_status.

g. Create an EXTERNAL table customer pointing to /user/clouduser/ecommerce/raw/customers with fields matching customers.csv.

h. Find the top 10 customer_city values by count of unique orders where order_status = 'delivered'.

SECTION 3 — ALTERYX [8 Marks]

a. Using Orders.csv and Payments.csv, build workflow Wf1_. Remove rows where order_status is NULL/empty; trim payment_type; replace NULL payment_installments with 1; identify Top 20 orders with highest payment_installments; output order_id, order_status, payment_type, payment_installments and payment_value to Top20_Orders_.csv.

b. Using Orders.csv, OrderItems.csv and Payments.csv, build workflow WF2_. Join the files on common columns; derive Order_value = price + shipping_charges; compute total order_value, total order_count and average payment_installments; output Revenue_summary_.

SECTION 4 — SPARK SQL [8 Marks]

a. Read orders, payments and customers from /user/clouduser/ecommerce/raw and register temporary views. For each customer_city calculate total orders, total payment_value and average payment_installments. Save as Parquet city_payment_summary under /user/clouduser/ecommerce/processed.

b. Read orders and customers and register temporary views. Join on customer_id to obtain customer_state. For each customer_state calculate avg_delivery_days and max_delivery_days. Use a Spark SQL Window Function to rank states from most delayed to least delayed as delay_rank. Flag avg_delivery_days > 15 as High Delay and others as On-Track. Save as Parquet delivery_analysis under /user/clouduser/ecommerce/processed.

SECTION 5 — SPARK STREAMING & KAFKA [10 Marks]

a. Create Kafka topic order-stream.

b. Read and publish orders.csv rows row-by-row to order-stream.

c. Verify delivery using the Kafka consumer console.

d. Create a Spark Structured Streaming source reading from order-stream.

e. Apply a Tumbling Window of 1 minute on the event timestamp and compute total orders in each order_status for each window.

f. Write the streaming aggregation to the console sink with outputMode = update and trigger every 30 seconds.

g. Feed data to the topic again. After at least 3 micro-batch outputs appear, stop the stream gracefully.

h. Additionally, write the streaming output to HDFS in Parquet format.

Assessment Deliverable: Execute task → capture command → capture output → document in Word → submit evidence.